

TREES

DAY 163
30 / Dec / 2025

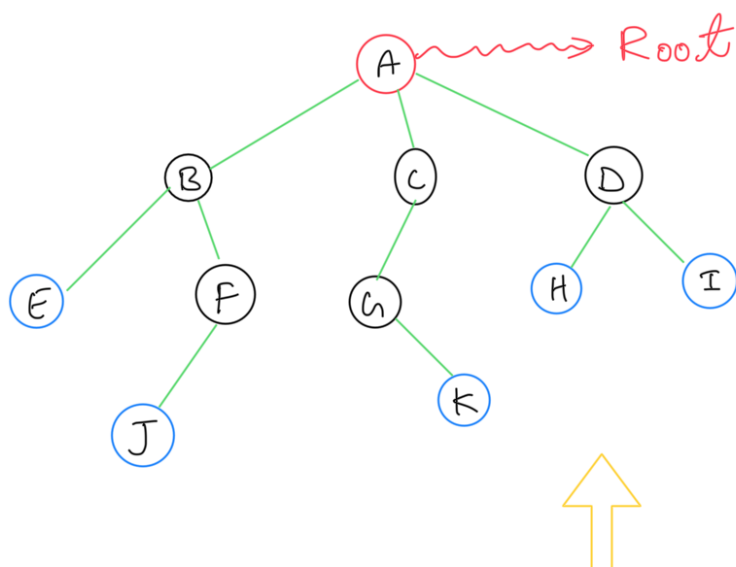
What is a tree?

A tree is a data structure similar to a linked list, but instead of each node pointing simply to the next node in a linear fashion, each node points to a number of nodes.

Tree is an example of Non-linear Data structure.

A tree structure is a way of representing the hierarchical nature of a structure in a graphical form.

NOTE: In trees ADT (Abstract Data Type), the order of the elements is not important, if we need ordering information, linear data structures like linked list, stacks, queues etc can be used.

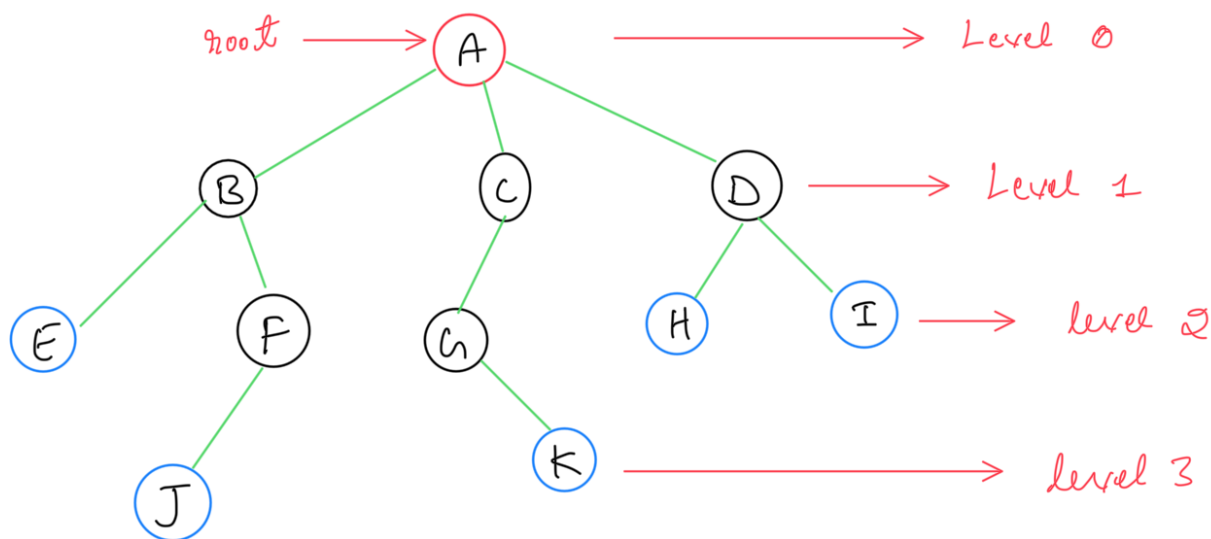


- root Node
- child Node / Parent Node
- edge
- Leaf Nodes

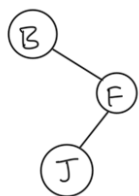
The root of a tree is the node with no parents. There can be at most one root node in a tree (node 'A' in the above example tree)

- A Edge refers to the link from parent to child
- A Node with no children is called leaf node (E, J, K, H and I)

- Children of Same parent are called Siblings (B, C, D are Siblings of A, and E, F are Siblings of B)
- A Node P is an ancestor of node Q if there exists a path from root to Q and P appears on the path.
- The node Q is called a descendant of P. (In Above Example A, C, and G are ancestors of K.
- The Set of all nodes at a given depth is called the level of the tree (B, C and D are the same level). The root node is at level Zero



- The depth of a node is the length of the path from the root to the node (depth of G is 2, $A \rightarrow C \rightarrow G$)
- The height of a node is the length of the path from that node to the deepest node. The height of a tree is the length of the path from the root to the deepest node in the tree.
A (rooted) tree with only one node (The root) has a height of Zero. In the previous example, the height of B is 2



- Height of the tree is the maximum height among all the nodes in the tree and depth of the tree is the maximum depth among all the nodes in the tree.

NOTE $\Rightarrow$ for a given tree, depth and height returns the Same Value, But for individual nodes we may get different results.

The Size of a Node is the Number of descendants it has including itself (the Size of the subtree C is 3)

✓ Advantages of Tree Data Structure

1. **Represents Hierarchical Data Naturally**
 - Perfect for parent-child relationships
 - Example: folders, organization charts
2. **Efficient Searching & Insertion**
 - Especially in structured trees (BST, B-Tree)
 - Faster than linear search in many cases
3. **Flexible & Dynamic**
 - Nodes can be added or removed easily
4. **Better Data Organization**
 - Keeps related data grouped logically
5. **Foundation for Advanced Algorithms**
 - Used in AI (decision trees), compilers, databases

✗ Disadvantages of Tree Data Structure

1. **More Memory Usage**
 - Extra pointers/references for child nodes
2. **Complex Implementation**
 - Harder than arrays or linked lists
3. **Traversal is Costly**
 - Requires recursion or stack (extra overhead)
4. **Balancing Issues**
 - Unbalanced trees can degrade performance ($O(n)$)
5. **Not Ideal for Simple Data**
 - Overkill for small or flat datasets

🏢 Where Trees Are Used (Very Important)

Area	Usage
File Systems	Folder & subfolder structure
Databases	B-Trees, B+ Trees for indexing
Compilers	Syntax & expression trees
Web	HTML DOM Tree
AI / ML	Decision Trees
Networking	Routing trees
OS	Process hierarchy

Node Structure (Multiple Children)

```
java

import java.util.ArrayList;
import java.util.List;

class TreeNode {
    int data;
    List<TreeNode> children;

    TreeNode(int data) {
        this.data = data;
        children = new ArrayList<>();
    }
}
```

- repeat (level) string " " repeats based on level

Output

code

```
1
 2
  5
  6
 3
 4
  7
```

Creating a Simple Tree

```
java

public class SimpleTree {

    public static void main(String[] args) {

        // Root node
        TreeNode root = new TreeNode(1);

        // Level 1 children
        TreeNode child1 = new TreeNode(2);
        TreeNode child2 = new TreeNode(3);
        TreeNode child3 = new TreeNode(4);

        root.children.add(child1);
        root.children.add(child2);
        root.children.add(child3);

        // Level 2 children
        child1.children.add(new TreeNode(5));
        child1.children.add(new TreeNode(6));

        child3.children.add(new TreeNode(7));

        // Print tree
        printTree(root, 0);

        // Recursive tree traversal
        static void printTree(TreeNode node, int level) {
            if (node == null) return;

            System.out.println(" ".repeat(level) + node.data);

            for (TreeNode child : node.children) {
                printTree(child, level + 1);
            }
        }
    }
}
```

Without Formatting Simple Traversing

```
java

void traverse(TreeNode node) {
    if (node == null) return;

    System.out.print(node.data + " ");

    for (TreeNode child : node.children) {
        traverse(child);
    }
}
```

1
2
5
6
3
4
7